

Advanced Python for Data Science

Week(3): GPU Programming & Performance

Tuesday, 30th June 2026, 14:00 - 17:15

Dr. Ghadeer Mobasher

Few Experts vs. An Army of Helpers

Imagine multiplying two matrices of size (10,000 x 10,000)

On your CPU: it may take 10 seconds and on your GPU, it will take a fraction of a second.
(Same Math, same Answer)

CPU: A handful of expert chefs

- A few highly skilled cores
- Each can handle complex, multi-step recipes
- Great at varied and branching tasks

GPU: An army of line chefs

- Thousands of simple cooks
- Each only chops one vegetable but all at once

Same Kitchen, a totally different staffing strategy.

- **Last week:** threads & processes shared the CPU's few expert chefs more cleverly.
- **This week:** we hire an entirely different kitchen staff, one built for one simple task repeated thousands of times: matrix math, the core of nearly all machine learning tasks.

Why do we need GPUs?

Last week's limit: Threads and processes only ever split work across a CPU's 4-16 cores.

What if we have workloads like matrix multiplication? Do we have enough parallelism?

A **GPU** trades a few powerful core for thousands of small, simple ones (ideal when the same operation must run on millions of data points)

Two Computation Models

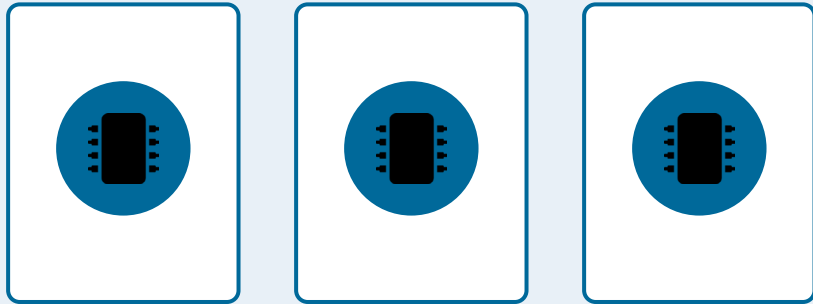
	CPU	GPU
Cores	4-16, powerful	Thousands, simple
Optimised for	Branching, varied logic	Same op, many data points
Best at	General-purpose code	Matrix / vector math

- *Both process data.*
- *They just specialise in opposite directions: depth of logic vs. breadth of repetition.*

Illustration of CPU cores vs. GPU cores

CPU

a few powerful cores

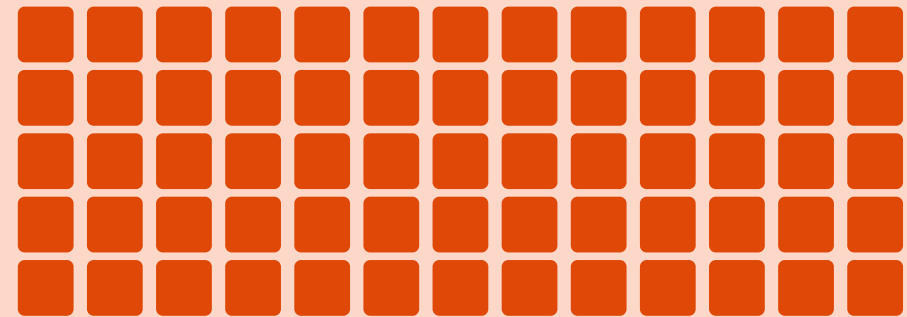


Each core: complex logic, branching, large cache

- Best for: general code, control flow, I/O
- A handful of “expert chefs” handling full recipes

GPU

thousands of lightweight cores



Each core: one simple operation, repeated in lockstep

- Best for: matrix / vector math, the same op on lots of data
- An army of “line cooks” each chopping one vegetable

One Operation, Millions of Times

The mental model

Data parallelism : Apply the exact same instruction to many independent pieces of data (at the same time).

```
for i in range(n):  
    c[i] = a[i] + b[i]  
# GPU: one thread handles each i,  
# all running simultaneously
```

- No element depends on another, perfectly parallelizable.
- This is exactly why vector/matrix math maps so well to GPUs

CPU vs. GPU on the Same Loop

CPU (sequential -ish)



one core works through elements, one (or a few) at a time

GPU (data -parallel)



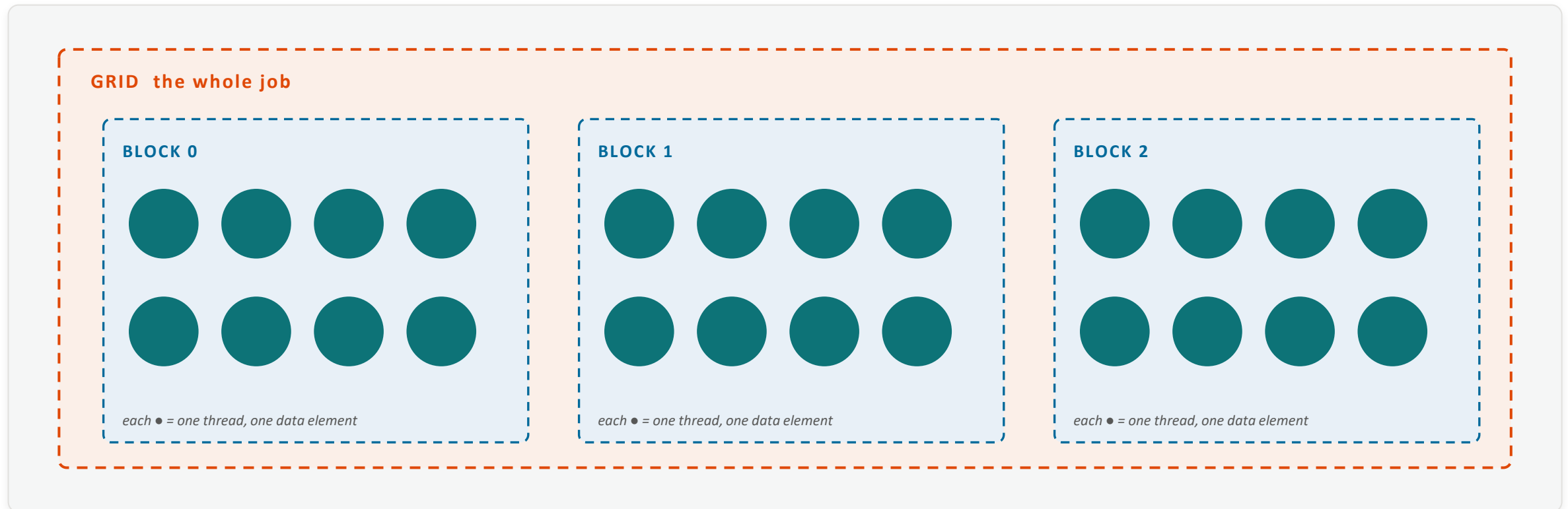
every element gets its own thread, all firing together

This is the single idea behind every GPU speed-up you'll see today.

GPU Execution Model

A **CUDA kernel** is the function the GPU runs; launching it spins up a **grid** of **blocks**, each containing many **threads**.
Every thread processes one piece of the data, all at once.

Grid → Block → Thread



GPU Execution Model

You rarely write CUDA directly



CUDA is NVIDIA's platform for writing GPU kernels in C/C++.

In Python, you almost never write raw CUDA, a library does it for you and exposes a NumPy-like interface.

Today's tool of choice: CuPy

Drop-in replacement for NumPy:

swap `np` for `cp` and most array code runs on the GPU unchanged.

Python GPU Library Landscape

- CuPy *NumPy compatible arrays on the GPU (Today's focus)*
- PyTorch *Tensors + autograd, deep learning training*
- TensorFlow *End-to-end ML framework, GPU/TPU support*

Same Code, Different Processor

First GPU Computations: Code observations

```
n = 20_000_000 # 20 million elements
# CPU – NumPy
# Create two random arrays of the given size
a_cpu = np.random.rand(n) # generates 20 million random
b_cpu = np.random.rand(n) floats uniformly distributed
                           in the range [0.0, 1.0)
t0 = time.perf_counter()
c_cpu = a_cpu + b_cpu
cpu_time = time.perf_counter() - t0

# GPU – CuPy (note: same syntax!)
a_gpu = cp.random.rand(n)
b_gpu = cp.random.rand(n)
t0 = time.perf_counter()
c_gpu = a_gpu + b_gpu
# Synchronize GPU to ensure all operations are complete
# before stopping the timer
# CuPy operations are asynchronous by default, so without
# this, timing would be inaccurate
cp.cuda.Stream.null.synchronize()
gpu_time = time.perf_counter() - t0
```

Notice anything?

The GPU version is almost line-for-line identical; swap np for cp. CuPy was designed so your NumPy intuition carries over directly.



The one extra line that matters

That synchronize() call isn't decoration, the next slide shows exactly why skipping it breaks your benchmark.

Your GPU Benchmark might be lying to you

First GPU Computations: Code observations

GPU kernel launches are asynchronous: the CPU hands work to the GPU and immediately moves to the next Python line **(It doesn't wait for the GPU to finish)** .

```
# GPU - WITHOUT sync
a_gpu = cp.random.rand(n)
b_gpu = cp.random.rand(n)
t0 = time.perf_counter()
c_gpu = a_gpu + b_gpu
gpu_time = time.perf_counter() - t0 # ↑ measures only the LAUNCH,
# not the actual computation
print(gpu_time)
```



0.00015269999857991934

Looks impossibly fast **(because you measured almost nothing)**

```
# GPU - WITH sync
a_gpu = cp.random.rand(n)
b_gpu = cp.random.rand(n)
t0 = time.perf_counter()
c_gpu = a_gpu + b_gpu
cp.cuda.Stream.null.synchronize() # ↑ blocks until the GPU is done
gpu_time = time.perf_counter() - t0
print(gpu_time)
```



0.21321010007523

Now gpu_time reflects what actually, happened on the device

Where the GPU pulls ahead

First GPU Computations: Code observations

```
def cpu_time(n):
    a = np.random.rand(n, n)
    b = np.random.rand(n, n)
    t0 = time.perf_counter()
    a @ b # dot operation
    return time.perf_counter() - t0

def gpu_time(n):
    a = cp.random.rand(n, n)
    b = cp.random.rand(n, n)
    t0 = time.perf_counter()
    a @ b
    cp.cuda.Stream.null.synchronize()
    return time.perf_counter() - t0
```

- Same helper-function pattern reused throughout the notebook
- Run for sizes 1000, 5000, 10000 and compare (Starter Code)

Matrix size vs. runtime

Small matrices: CPU can still win once GPU overhead is included.

Mid-size matrices: GPU's parallel cores start to dominate.

Large matrices: GPU advantage is largest (thousands of cores finally have enough work to stay busy)

The crossover point where GPU starts winning is exactly what you'll measure and plot in this week's lab.

The GPU's speed doesn't include the trip there

Memory Management: Code observations

```
x_cpu = np.random.rand(5000, 5000)

t0 = time.perf_counter()
x_gpu = cp.asarray(x_cpu)      # H→D
cp.cuda.Stream.null.synchronize()
t1 = time.perf_counter()

y_gpu = x_gpu * 2              # compute
cp.cuda.Stream.null.synchronize()
t2 = time.perf_counter()

y_cpu = cp.asnumpy(y_gpu)      # D→H
t3 = time.perf_counter()
```

Host memory (RAM, CPU-side) and **device memory** (VRAM, GPU-side) are physically separate:

→ data must be copied across the PCIe bus to move between them.

Three Timed Stages

● Transfer in	$t_1 - t_0$	CPU → GPU
● Compute	$t_2 - t_1$	on the GPU
● Transfer out	$t_3 - t_2$	GPU → CPU

For small arrays, transfer in + transfer out can outweigh the compute speed -up entirely; the GPU finishes the math fast, but the round-trip costs more than the CPU would have taken alone.

Who decides when Data moves?

Memory Management: Code observations

Implicit Transfer

```
x_gpu = cp.array(x_cpu)
```

- *Convenient*, CuPy handles the copy for you
- But the moment of transfer is hidden inside the call
- Easy to accidentally re-transfer the same data repeatedly in a loop

Explicit Transfer

```
x_gpu = cp.asarray(x_cpu, copy =True)
```

- You control exactly when the copy happens
- Makes repeated computations efficient
(transfer once, reuse the GPU array many times)
- Recommended once you're optimising a real pipeline

Rule of thumb: if data crosses the CPU ↔ GPU boundary **more than once** , transfer it explicitly and keep it on the GPU between operations.

A Third option : TPUs

Hardware Ecosystem

	CPU	GPU	TPU
	<i>General-purpose, branching logic</i>	<i>Parallel math, thousands of cores</i>	<i>Tensor ops only, built for ML</i>
Flexibility	Highest	Medium	Lowest (fixed ops)
Best at	Everything	Matrix / vector math	Matrix multiply + activation, at huge scale
Typical use	App logic, orchestration	Training & inference	Very large-scale training (Google Cloud)

Switching to a TPU in Colab

Runtime → Change runtime type → Hardware accelerator → TPU.

Team C

15-20 minutes presentation

Team Maverick

15-20 minutes presentation

Pause / Break

See you in 15 minutes 😊!
Please be on time..

Exercises



Solve Problems (1 -4) in “Week3_Exercise.ipynb “ and you have 45 minutes to finish all the tasks. *Don't forget to split your screen. Read the whole instructions first and answer the reflection questions in your own words.*

Problem 1 : Verify GPU Access

Objective: Confirm that the Colab environment provides a GPU and understand its capabilities.

Tasks:

1. Enable GPU runtime in Google Colab.
2. Run:

```
!nvidia-smi
```

3. Record the GPU model, memory, and CUDA version.

Reflection Questions:

- What GPU model was allocated?
- How does the GPU compare to your CPU in terms of cores and memory?

Problem 2: CPU vs GPU Vector Operations

Objective: Compare simple numerical operations on CPU vs GPU.

Tasks:

1. Create two large vectors (e.g., size 20,000,000) using **NumPy**.
2. Perform vector addition on the CPU and measure runtime.
3. Repeat the operation on the GPU using **CuPy**.
4. Compare the execution times.

Reflection Questions:

- Which operation was faster and why?
- For smaller arrays, would GPU still be faster?
- How does asynchronous execution affect GPU timing?

Problem 3 : Measuring Data Transfer Costs

Objective: Quantify the cost of moving data between CPU and GPU memory.

Tasks:

1. Transfer a large array from CPU to GPU.

Please submit your in -class exercises via Moodle.

Take-Home Assignment

Complete the following challenge at home. Remember to split your screen and solve the problems. You may always Google, check the syntax, resolve errors, and brainstorm, but **do not generate code or answers**. See the [submission instructions](#).

Mini Challenge: Investigating GPU Performance Bottlenecks

Objective

The objective of this mini challenge is to analyse and compare CPU and GPU execution behaviour across different computational patterns. You will profile several workloads and determine the underlying performance characteristics.

You are expected to identify whether each workload is:

- memory-bound
- compute-bound
- dominated by kernel launch overhead
- affected by repeated CPU-GPU synchronisation

You should justify all observations using profiling results.

General Instructions

For all tasks:

1. Measure execution time using `time.perf_counter()`.
2. Ensure GPU correctness by using:

```
cp.cuda.Stream.null.synchronize()
```

Before stopping the timer, do the following:

3. Run each experiment at **least three times** and consider the **average value**.
4. Keep array sizes unchanged unless explicitly instructed.
5. Record both CPU and GPU execution times where applicable.

- The deadline is 05.07.2026 @ 23:59.
- See you next time when we dive into **Software Design & Testing**

Key Takeaways

Architecture

CPU: few powerful cores. GPU: thousands of simple ones.

Execution Model

Kernels run across a Grid of Blocks of Threads; one thread per data element.

Libraries

CuPy mirrors NumPy syntax; PyTorch/TF cover other GPU needs.

Async Timing

Always synchronize() before stopping the clock on GPU code.

Memory Cost

Transfer in/out can outweigh compute gains for small data.

Hardware Choice

CPU, GPU, or TPU, match the tool to the workload's shape.

Any Questions?

See you next time 😊

Dr. Ghadeer Mobasher

ghadeer.mobasher@srh-hochschulen.de

<https://www.linkedin.com/in/ghadeermobasher/>

LGS 6, 2. OG, Arc 213

